COMPREHENSIVE ASSIGNMENT

GRADIENT BOOSTING

Jaineet Shah
Computer Science Department
Nirma University
Ahmedabad, India
18bce083@nirmauni.ac.in

1. INTRODUCTION -

The concept of boosting originated from a question, that whether a weak learner can be improved. Later on Jerome H. Friedman developed the gradient boosting algorithm. Gradient Boosting is a machine learning technique which produces a prediction model based on the intuition that many weak prediction models can be combined to create a powerful prediction model. A weak learner or model is defined as one whose performance is just better than a random chance. The most widely used weak learners are decision trees. Gradient Boosting technique consists of three key elements,

a) Loss function
b) Weak learner
c) Additive model to combine weak learners

The loss function depends on the type of problem and it must be differentiable. For example - Mean Squared Error for Regression, Logarithmic loss for Classification. One can also use a self-developed loss function.

As mentioned above, decision trees or regression trees are used as weak learners. The decision trees which are used to split continuous values are known as regression trees. The trees are built in a greedy fashion, i.e. ,choosing the best partitions based on purity scores like information gain or the minimisation of the loss function. Aspects of the decision tree such as number of nodes, layers or partitions are restricted generally in order to maintain the weakness of the learners.

In an Additive model, the trees are added one at a time and the already added trees are left unchanged. Gradient descent procedure is followed to minimise the loss when the trees are added. Normally, gradient descent is used to minimise a set of parameters but in case of gradient boosting, the tree is parameterised and hence gradient descent is used to update the parameters of the tree in order to minimise the loss. This form of gradient descent is known as functional gradient descent.

2. GRADIENT BOOSTING vs ADAPTIVE BOOSTING -

Gradient Boosting is very similar to Adaptive Boosting or AdaBoost.

In order to predict a continuous or a categorical attribute, AdaBoost begins by creating a very short decision tree known as stump from the training data. The degree of significance a stump has on the final result depends on how well it compensated for the previous errors. Then it creates the next stump based on the errors made by the previous stump. The algorithm goes on until the model has a perfect fit or a certain amount of stumps are created.

Gradient Boost begins by forming a single leaf instead of a stump. The leaf acts as a guess representing the values of all the samples. Then the gradient boost algorithm builds a tree and just like AdaBoost, the tree formed is based on the errors made by the previous tree. But unlike AdaBoost, the formed tree is generally larger than a stump. However, the size of the tree is restricted. In practice, the maximum number of leaves are most often set to be between 8 and 32. Also the formed trees can be different every time.

Therefore, just like AdaBoost, Gradient Boost creates fixed size trees based on the previous tree's errors but unlike AdaBoost the size of the tree can larger than a stump. Also like AdaBoost, Gradient Boost scales the trees, but it scales the trees by the same amount.

3. REGRESSION USING GRADIENT BOOSTING -

The Gradient Boost Regression algorithm starts by making an initial guess(prediction) which represents all the samples of the training dataset. The initial guess is equal to the mean value of all the samples of the dependent variable in the training dataset provided the loss function used is one-half mean squared error. This mean value is taken as a leaf node. The next step is to create a tree based on the errors made by the previous tree. The errors made by the previous tree are the differences between the actual values and the predicted values. These difference are also known as Pseudo Residuals. After the calculation of the pseudo residuals, a tree is built using the independent attributes of the dataset with any greedy approach like Gini index to predict the pseudo residuals instead of the actual values of the dependent variable. If the newly built tree contains a leaf which possesses more than one value then the output value of that particular leaf is the mean of those values provided that one-half mean squared error is taken as the loss function. After the creation of a new tree, all the previously built trees are combined with the new tree to form a single prediction model. Then the above mentioned procedure is iterated again to form a new tree based on the errors made by the previous prediction model until a certain number of trees are built or the creation of more number of trees no longer improves the fitness of the model. Also to counter the variance in the model and for better prediction with a testing dataset, Gradient Boosting uses a leaning rate generally between 0 and 1 to scale the contribution of any newly built tree after the first leaf node in the prediction model.

ALGORITHM -

TRAINING SET and LOSS FUNCTION

Input: Data $\{(x_i, y_i)\}_{i=1}^{n}$, and a differentiable **Loss Function** $L(y_i, F(x))$

x_i refers to each row of the independent attributes in the training dataset.
y_i refers to each row the dependent attribute in the training dataset.
i ranges from 1 to n where n is the total number of samples in the training dataset.
L is the Loss function which is assumed to be one-half mean squared error.(Most Popular)
F(x) is a function which returns the predicted values.

*Since the loss function = (1/2) * (Actual - Predicted)^2*
The derivative of the loss function with respect to predicted value can be written as -(Actual - Predicted).

STEP - 1

Initialize model with a constant value: $F_0(x) = \underset{\gamma}{\operatorname{argmin}} \sum_{i=1}^{n} L(y_i, \gamma)$

γ refers to the predicted values.
argmin means that $F_0(x)$ must be assigned the predicted value which minimises the Σ (summation).
$F_0(x)$ can be calculated by using the derivative of the Σ of the loss function and then equating it to zero which in turn gives the mean of the actual values of the dependent variable.
This initial predicted value is taken as the first leaf node in the Gradient Boost algorithm.

STEP - 2

for m = 1 to M:

(A) Compute $r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F(x)=F_{m-1}(x)}$ for $i = 1,\ldots,n$

(B) Fit a regression tree to the r_{im} values and create terminal regions R_{jm}, for $j = 1\ldots J_m$

(C) For $j = 1\ldots J_m$ compute $\gamma_{jm} = \underset{\gamma}{\operatorname{argmin}} \sum_{x_i \in R_{ij}} L(y_i, F_{m-1}(x_i) + \gamma)$

(D) Update $F_m(x) = F_{m-1}(x) + \nu \sum_{j=1}^{J_m} \gamma_m I(x \in R_{jm})$

Step - 2 is a loop in which all the trees are built.M is the certain fixed number of trees to be built. Generally M is taken as 100.
m represents a single tree. E.g. - m = 5 represents the fifth tree.

Part (A) refers to the derivative(gradient) of the loss function with respect to the predicted value which is equal to -(Actual - Predicted) as mentioned above. Since there is a negative in front of the derivative, r_{im} is equal to (Actual - Predicted) which is in fact the pseudo residual. $F_{m-1}(x)$ is the function for the predicted values given by the previous tree. r in r_{im} is short for residual.

Part (B) states that a regression tree should be built to predict the pseudo residuals and not the actual values of the dependent variable. The regression tree is built using the independent attributes with a greedy approach. The terminal region means the leaf node. j is the index for each leaf node in the tree.

Part (C) determines the output values for each leaf node of the regression tree built in Part (B). The output value for each leaf node is the value of γ which minimises the Σ. This part is the same as Step-1 with just a minor difference, i.e. , the predicted values of the previous tree is taken into account ($F_{m-1}(x_i) + \gamma$) and also Σ includes only the selected samples ($x_i \varepsilon R_{jm}$). Also, the calculation of the output value in turn gives the mean of the pseudo residuals of the selected samples in the regression tree built in Part (B).

Part (D) makes a new prediction for each input row. The new prediction is based on the combination of the first initial leaf node and all the regression trees made previously. Σ is present to dish out the single output value of interest from all the other values. ν is the learning rate.

Upon completion of all the iterations of Step-2, $F_M(x)$ is found which is the final output of the Gradient Boost algorithm.

4. CLASSIFICATION USING GRADIENT BOOSTING -

The procedure for classification using Gradient Boosting and regression using Gradient Boosting is very similar. Just like in regression, the Gradient Boost classification algorithm starts by making an initial guess(prediction) representing all the samples of the training dataset. The following procedure is described under the assumption that there are only 2 classes present for classification. The initial guess can be taken as the log(odds) where the odds that the sample belongs to class-1 is the number of samples belonging to class-1 divided by the number of samples belonging to class-2. This initial guess is taken as a leaf node. One of the simplest ways to use log(odds) for classification is by transforming it into probability using the logistic function. Then a threshold needs to be decided for making the classification decisions based on probability. 0.5 is a very common threshold for binary classification. If 0.5 is taken as the threshold then it can be assumed that if the probability of a sample is above 0.5, it belongs to class-1 and class-2 if vice-versa. The next step is to calculate the pseudo residuals, i.e. , (Actual - Predicted) where the actual value for class-1 is 1 and for class-2 is 0. A tree is then built to predict the residuals using the independent attributes with a greedy approach. In case of the Gradient Boost regression algorithm, the output value of a leaf node was equal to the mean value of the residuals but for Gradient Boost classification algorithm the procedure is not that simple because the predictions are in terms of log(odds) but the tree is built using probability. Therefore a transformation is required to calculate the output value of a leaf node. One of the most common transformation used is,

$$\frac{\sum \text{Residual}_i}{\sum \left[\text{Previous Probability}_i \times (1 - \text{Previous Probability}_i)\right]}$$

The numerator is the sum of all the residuals of the leaf node and the denominator is the sum of previously predicted probability for each residual multiplied by one minus the same predicted probability. The next step is to make new predictions by combining all the previously built trees including the initial leaf node. Just like in regression, the new trees are also scaled by a learning rate. Since the predictions are in log(odds), they are again transformed into probability using the Logistic function. The whole procedure is repeated again until a certain number of trees are built or the creation of more number of trees no longer improves the fitness of the model.

5. CONCLUSION -

This paper has thoroughly reviewed the concept of gradient boosting, Gradient Boost regression algorithm, Gradient Boost classification algorithm, comparison between AdaBoost and Gradient Boost and the origin of boosting .